

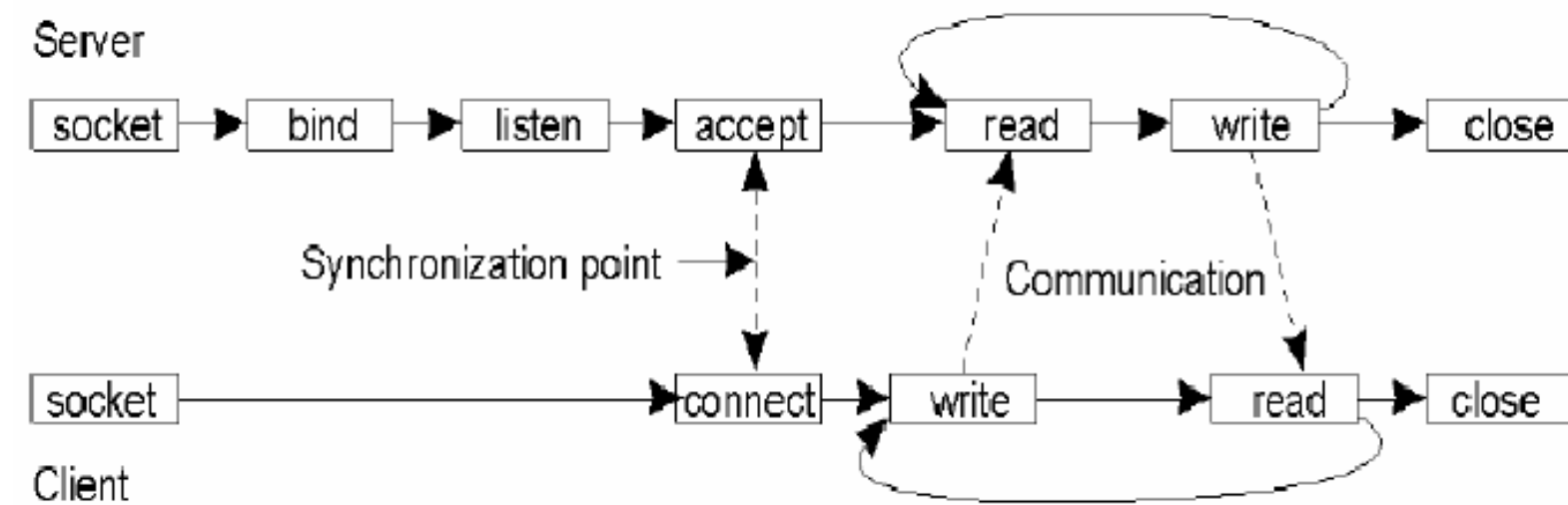
Sockets

Programação Orientada a objetos

Funcionamento

- O **Servidor** ficará a **espera** de **ligações**;
- O **cliente** irá **ligar-se** ao **servidor**, **estabelecendo** uma **conexão**;
- Através desta conexão é possível uma **comunicação bidirecional**;
- O **socket** irá representar o **extremo** da **conexão**;
- A **conexão** é caracterizada pelo **par de sockets cliente e servidor**;
- A **comunicação** é feita através da **troca de mensagens**, sendo que tais mensagens podem ser **strings, inteiros, floats, objetos...**

Funcionamento básico



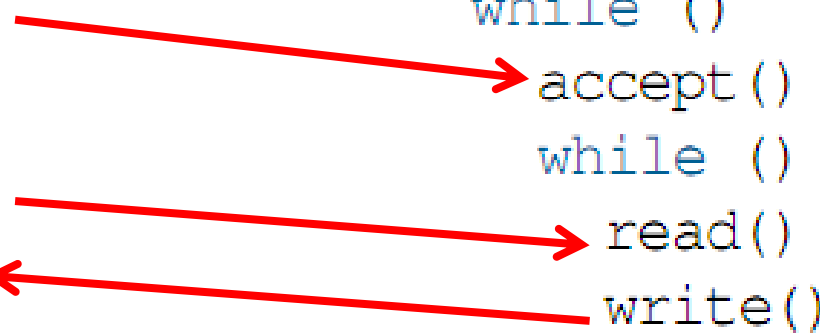
Funcionamento básico

Cliente:

```
socket()  
connect()  
while ()  
    write()  
    read()  
close()
```

Servidor

```
socket()  
bind()  
listen()  
while ()  
    accept()  
    while ()  
        read()  
        write()  
close()
```



Java Sockets

- Servidor:
 1. Cria um **ServerSocket**;
 2. **Escuta** no socket através do método **accept()**; a **espera** de uma tentativa de **conexão**;
 3. Uma vez estabelecida a conexão, **envia e recebe** fluxos de **dados**;
 4. Quando a **conexão** for **concluída**, **fecha** a conexão.
 5. Geralmente **retorna** ao **passo 2**;

Java Sockets

- Cliente:
 1. Cria um **socket** através do construtor da **classe Socket**;
 2. Tenta **estabelecer** uma **conexão** com o servidor;
 3. Uma vez estabelecida a conexão, **envia e recebe** fluxos de **dados**;
 4. Quando a conexão for concluída, **fecha a conexão**.

Java Sockets

- **Classe ServerSocket** (métodos)
 - `ServerSocket(int port)`: cria um servidor socket, limitado a uma porta específica;
 - `Socket accept()`: fica escutando por uma conexão com este socket e aceita ela;
 - `void close()`: fecha o socket;
 - `InetAddress getInetAddress()`: retorna o endereço local deste socket servidor.

Java Sockets

- **Classe Socket** (métodos)
 - **Socket(InetAddress address, int port)**: cria um socket e conecta ele a um número de porta específico em um determinado endereço IP;
 - **void close()**: fecha o socket;
 - **InetAddress getInetAddress()**: retorna o endereço no qual o socket está conectado;
 - **boolean isClosed()**: retorna verdadeiro caso o socket esteja fechado e falso caso contrário;
 - **boolean isConnected()**: retorna o estado de conexão do socket.

Java Sockets

- Envio e recebimento de dados
 - Os dados são enviados através de fluxos de dados de entrada e saída;
 - Os métodos:
 - `InputStream getInputStream()`: retorna um fluxo de entrada para este socket;
 - `OutputStream getOutputStream()`: retorna um fluxo de saída para este socket

Servidor

```
10 public static void main(String[] args) {
11     try {
12         //Criação do socket servidor e definição da porta (1234)
13         ServerSocket servidor = new ServerSocket(1234);
14         System.out.println("Socket servidor criado com sucesso");
15
16         while (true) {
17             /*Servidor aguarda uma conexão de um cliente
18             O servidor bloqueia nesta linha enquanto um cliente não chega
19             Quando um cliente se conecta, o servidor recebe uma referência do
20             socket do cliente que se conectou (retorno do método)
21             */
22             Socket cliente = servidor.accept();
23             /* Extraio o fluxo de saída e entrada de dados do socket */
24             DataOutputStream out = new DataOutputStream(cliente.getOutputStream());
25             DataInputStream in = new DataInputStream(cliente.getInputStream());
26
27             /* Servidor recebe 2 inteiros do cliente, x e y */
28             int x, y, resposta;
29             x = in.readInt();
30             y = in.readInt();
31             System.out.println("Recebi " + x + " e " + y);
32             /* Servidor calcula a resposta e envia para o cliente */
33             resposta = x + y;
34             out.writeInt(resposta);
35
36             cliente.close();
37         }
38     } catch (Exception e) {
39         e.printStackTrace();
40     }
41 }
42 }
```

```
10 public static void main(String[] args) {
11     try {
12         //Criação do socket servidor e definição da porta (1234)
13         ServerSocket servidor = new ServerSocket(1234);
14         System.out.println("Socket servidor criado com sucesso");
15
16         while (true) {
17             /*Servidor aguarda uma conexão de um cliente
18             O servidor bloqueia nesta linha enquanto um cliente não chega
19             Quando um cliente se conecta, o servidor recebe uma referência do
20             socket do cliente que se conectou (retorno do método)
21             */
22             Socket cliente = servidor.accept();
23             /* Extraio o fluxo de saída e entrada de dados do socket */
24             DataOutputStream out = new DataOutputStream(cliente.getOutputStream());
25             DataInputStream in = new DataInputStream(cliente.getInputStream());
26
27             /* Servidor recebe 2 inteiros do cliente, x e y */
28             int x, y, resposta;
29             x = in.readInt();
30             y = in.readInt();
31             System.out.println("Recebi " + x + " e " + y);
32             /* Servidor calcula a resposta e envia para o cliente */
33             resposta = x + y;
34             out.writeInt(resposta);
35
36             cliente.close();
37         }
38     } catch (Exception e) {
39         e.printStackTrace();
40     }
41 }
42 }
```

Cliente

```
7 public class Cliente {
8     public static void main(String[] args) {
9         try {
10             /*
11              Criação de um socket cliente e tentativa de conexão
12              no ip "localhost", porta 1234
13              */
14             Socket s = new Socket("localhost", 1234);
15             System.out.println("Conexão estabelecida com sucesso");
16
17             /*
18              Extraio o fluxo de saída e entrada de dados do socket
19              */
20             DataOutputStream out = new DataOutputStream(s.getOutputStream());
21             DataInputStream in = new DataInputStream(s.getInputStream());
22
23             int x = 5, y = 6, resposta;
24             /*
25              Envia o x e o y para o servidor
26              */
27             out.writeInt(x);
28             out.writeInt(y);
29             /*
30              Aguardo a resposta do servidor
31              */
32             resposta = in.readInt();
33
34             System.out.println("A soma é "+resposta);
35
36             s.close();
37
38         } catch (Exception e) {
39             e.printStackTrace();
40         }
41     }
42 }
```

```
7 public class Cliente {
8     public static void main(String[] args) {
9         try {
10             /*
11              Criação de um socket cliente e tentativa de conexão
12              no ip "localhost", porta 1234
13              */
14             Socket s = new Socket("localhost", 1234);
15             System.out.println("Conexão estabelecida com sucesso");
16
17             /*
18              Extraio o fluxo de saída e entrada de dados do socket
19              */
20             DataOutputStream out = new DataOutputStream(s.getOutputStream());
21             DataInputStream in = new DataInputStream(s.getInputStream());
22
23             int x = 5, y = 6, resposta;
24             /*
25              Envia o x e o y para o servidor
26              */
27             out.writeInt(x);
28             out.writeInt(y);
29             /*
30              Aguardo a resposta do servidor
31              */
32             resposta = in.readInt();
33
34             System.out.println("A soma é "+resposta);
35
36             s.close();
37
38         } catch (Exception e) {
39             e.printStackTrace();
40         }
41     }
42 }
```

Sockets – Envio e recebimento de objetos

Sockets – Envío de objetos

```
5 public class Numero implements Serializable {
6
7     private int x, y;
8
9     public Numero(int x, int y) {
10         this.x = x;
11         this.y = y;
12     }
13
14     public int getX() {
15         return x;
16     }
17
18     public void setX(int x) {
19         this.x = x;
20     }
21
22     public int getY() {
23         return y;
24     }
25
26     public void setY(int y) {
27         this.y = y;
28     }
29 }
```

Sockets – Resposta de objetos

```
5 public class Resposta implements Serializable {  
6     private int valor;  
7  
8     public Resposta(int valor) {  
9         this.valor = valor;  
10    }  
11  
12    public int getValor() {  
13        return valor;  
14    }  
15  
16    public void setValor(int valor) {  
17        this.valor = valor;  
18    }  
19  
20 }
```


Sockets – Servidor de recebimento e envio de objetos

```
10 public static void main(String[] args) {
11     try {
12         //Criação do socket servidor e definição da porta (1234)
13         ServerSocket servidor = new ServerSocket(1234);
14         System.out.println("Socket servidor criado com sucesso");
15
16         while (true) {
17             /*Servidor aguarda uma conexão de um cliente
18             O servidor bloqueia nesta linha enquanto um cliente não chega
19             Quando um cliente se conecta, o servidor recebe uma referência do
20             socket do cliente que se conectou (retorno do método)
21             */
22             Socket cliente = servidor.accept();
23             /* Extraio o fluxo de saída e entrada de objetos do socket */
24             ObjectOutputStream out = new ObjectOutputStream(cliente.getOutputStream());
25             ObjectInputStream in = new ObjectInputStream(cliente.getInputStream());
26
27             Numero n;
28             Resposta res;
29             /* Servidor recebe 1 objeto do tipo Numero do cliente */
30             n = (Numero) in.readObject();
31
32             System.out.println("Recebi " + n.getX() + " e " + n.getY());
33             /* Servidor calcula a resposta e envia para o cliente */
34             res = new Resposta(n.getX()+n.getY());
35             out.writeObject(res);
36
37             cliente.close();
38         }
39     } catch (Exception e) {
40         e.printStackTrace();
41     }
42 }
43 }
```

```
10 public static void main(String[] args) {
11     try {
12         //Criação do socket servidor e definição da porta (1234)
13         ServerSocket servidor = new ServerSocket(1234);
14         System.out.println("Socket servidor criado com sucesso");
15
16         while (true) {
17             /*Servidor aguarda uma conexão de um cliente
18             O servidor bloqueia nesta linha enquanto um cliente não chega
19             Quando um cliente se conecta, o servidor recebe uma referência do
20             socket do cliente que se conectou (retorno do método)
21             */
22             Socket cliente = servidor.accept();
23             /* Extraio o fluxo de saída e entrada de objetos do socket */
24             ObjectOutputStream out = new ObjectOutputStream(cliente.getOutputStream());
25             ObjectInputStream in = new ObjectInputStream(cliente.getInputStream());
26
27             Numero n;
28             Resposta res;
29             /* Servidor recebe 1 objeto do tipo Numero do cliente */
30             n = (Numero) in.readObject();
31
32             System.out.println("Recebi " + n.getX() + " e " + n.getY());
33             /* Servidor calcula a resposta e envia para o cliente */
34             res = new Resposta(n.getX()+n.getY());
35             out.writeObject(res);
36
37             cliente.close();
38         }
39     } catch (Exception e) {
40         e.printStackTrace();
41     }
42 }
43 }
```

Sockets – Cliente de envio e recebimento de objetos

```
7 public class Cliente {
8
9     public static void main(String[] args) {
10         try {
11             /* Criação de um socket cliente e tentativa de conexão
12              no ip "localhost", porta 1234 */
13             Socket s = new Socket("localhost", 1234);
14             System.out.println("Conexão estabelecida com sucesso");
15
16             /* Extraio o fluxo de saída e entrada de objetos do socket */
17             ObjectOutputStream out = new ObjectOutputStream(s.getOutputStream());
18             ObjectInputStream in = new ObjectInputStream(s.getInputStream());
19
20             Numero n = new Numero(12, 6);
21             Resposta res;
22             /* Envia o objeto n para o servidor */
23             out.writeObject(n);
24             /* Aguardo a resposta do servidor */
25             res = (Resposta)in.readObject();
26
27             System.out.println("A soma é "+res.getValor());
28
29             s.close();
30
31         } catch (Exception e) {
32             e.printStackTrace();
33         }
34     }
35 }
```

```
7 public class Cliente {
8
9     public static void main(String[] args) {
10         try {
11             /* Criação de um socket cliente e tentativa de conexão
12              no ip "localhost", porta 1234 */
13             Socket s = new Socket("localhost", 1234);
14             System.out.println("Conexão estabelecida com sucesso");
15
16             /* Extraio o fluxo de saída e entrada de objetos do socket */
17             ObjectOutputStream out = new ObjectOutputStream(s.getOutputStream());
18             ObjectInputStream in = new ObjectInputStream(s.getInputStream());
19
20             Numero n = new Numero(12, 6);
21             Resposta res;
22             /* Envia o objeto n para o servidor */
23             out.writeObject(n);
24             /* Aguardo a resposta do servidor */
25             res = (Resposta)in.readObject();
26
27             System.out.println("A soma é "+res.getValor());
28
29             s.close();
30
31         } catch (Exception e) {
32             e.printStackTrace();
33         }
34     }
35 }
```

Atividade final

- Parte 2
 - Utilizando a base desenvolvida de cadastro de veículos, agora implemente uma aplicação cliente x servidor
 - No cliente é onde ficará a lista e o arquivo de veículos cadastrado.
 - A aplicação cliente não terá acesso a lista e deverá ler os dados de um veículo para ser cadastrado.
 - A aplicação cliente deverá instanciar um objeto de veículo e enviar um objeto para o servidor para ser cadastrado
 - No servidor, deverá ser cadastrado e deverá retornar para o cliente se o cadastro foi efetuado com sucesso ou não